GridSync

AI-Driven AWS Cloud Infrastructure for Smart EV Fast-Charging and Power Grid Management

A Case Study and Implementation Report

Cloud Architecture Design · Final Project

Team 4	
Anish P	24BCE2394
Neeraj K	24BCE2393

Deployed on Amazon Web Services, region ap-southeast-2 (Sydney)
Live dashboard: https://dye305b97k5t7.cloudfront.net
Source code: github.com/Anish-p-creator/Cloud-power-grid-management

Contents

Abstract

1. Introduction: a real-time problem

2. Problem statement: four pillars

3. Objectives

4. Literature survey (2020 to 2026)

5. Analysis of existing research and research gap

6. Challenges

7. AWS services used

8. Proposed and deployed architecture

9. AI and machine-learning models

10. Implementation on AWS

11. Security design

12. Cost analysis

13. Results and evaluation

14. Novelty

15. Limitations and future work

16. Conclusion

References

Appendix A: repository layout and how to run

Abstract

Evening fast-charging peaks, with many 150 to 350 kW chargers ramping up on the same distribution transformer, overload local grid equipment and cause breaker trips. The underlying issue is coordination rather than energy supply: utilities signal demand response with OpenADR, chargers are controlled with OCPP, and existing back-ends take 15 to 30 seconds to react. This project designs and deploys **GridSync**, a fully serverless AWS platform that bridges the two protocols and recomputes every charger's power limit every two seconds. Decisions are made by three machine-learning models implemented from scratch: a neural-network demand forecaster, a Q-learning reinforcement-learning charging agent, and an Isolation Forest intrusion detector that revokes a compromised charger's identity in AWS IoT Core. The system runs on a real AWS account as a single CloudFormation stack of 30 resources (IoT Core, SQS, Lambda, Step Functions, API Gateway, DynamoDB, S3, CloudFront, IAM, CloudWatch and Budgets). Measured live, a control decision takes about **0.16 s**, a utility OpenADR request reaches all chargers in **0.55 s** (target under 2 s), and a hacked charger is detected and isolated within about one second, with zero feeder trips in AI mode.

1. Introduction: a real-time problem

At a highway rest stop or city fast-charging hub around 6 or 7 PM, five to ten EVs are often plugged in at once, each drawing 150 to 350 kW from the same local feeder. Each charger is rated for that power, but the transformer and substation feeding the site were never designed to carry the combined load at the same peak hour every day.

When several ultra-fast chargers ramp up together, the current drawn from the local distribution transformer spikes. If the transformer is already carrying neighbourhood load (air conditioners, shops closing, streetlights switching on), the extra EV load pushes it past its rating. Voltage sags, and protection relays and breakers trip, taking the whole feeder offline until someone resets it.

The core of the problem is **timing and coordination**, not raw power availability. Dozens of chargers switch on and off independently with no shared, real-time view of how much headroom the transformer has left. This project builds a cloud system that gives the grid and the chargers a fast, shared and secure way to coordinate *before* a trip happens.

2. Problem statement: four pillars

Pillar	What goes wrong
1. Protocol disconnect	Utilities use OpenADR (price and load-shedding signals); chargers use OCPP (sessions, meter values). No native real-time bridge exists, so every deployment relies on one-off glue code.
2. Latency at scale	Traditional web back-ends queue up when many chargers report at once; deciding and pushing an action can take 15 to 30 s, while protection equipment reacts in a fraction of that.
3. Inefficient static allocation	Each charger has a fixed power ceiling set at installation, ignoring time of day, grid state and each car's state of charge.
4. Vulnerable edge security	Chargers sit in public places on cellular links, often with weak authentication. A compromised charger is an entry point to systems that talk to the grid.

Table 1: The four pillars of the fast-charging overload problem

Together these show that installing more chargers or writing more integration code will not fix overloads. What is needed is a purpose-built cloud platform that speaks both protocols, reacts in near real time, allocates power dynamically and treats security as a first-class requirement.

3. Objectives

#	Objective	Measure
O1	Fast reaction during grid stress	New limits pushed to chargers in under 2 seconds
O2	Lower peak demand	Reduce the site's EV demand peak versus an uncoordinated baseline (report target: at least 35% lower demand charges)
O3	Protect the transformer	Load kept below 100% of rating, zero tripping events
O4	High availability	99.99% cloud uptime for charger connections
O5	Zero-trust security	Every device, message and service call authenticated; compromised devices isolated

Table 2: Project objectives

4. Literature survey (2020 to 2026)

Ten works relevant to cloud infrastructure for EV charging, dynamic load balancing, cloud battery management and grid coordination were reviewed.

Work	Architecture	Focus	Advantage / disadvantage
Gao et al. (2020): remote monitoring and fault detection for EV charging devices	Centralised cloud streaming of charger data	Preventive maintenance, fault detection	+ Proved cloud streaming of telemetry. – Narrow fault logic, no reaction to grid load
Concept review of a cloud-based smart BMS, MDPI Batteries (2022)	Split between local BMS and cloud compute	Limited on-board compute and storage	+ Clear cloud vs local framework. – Conceptual, no deployment
Karnehm et al. (2023): cloud framework for battery analytics	Cloud telemetry with Extended Kalman Filter SoC estimation	Fleet-scale real-time SoC	+ Moved to predictive analytics. – EKF assumptions break for aged batteries
Saner, Saha and Srinivasan (2023), IEEE T-ITS: charge-curve-aware scheduling	Per-vehicle condition-aware scheduling	Optimal scheduling at one site	+ Dynamic per-bay limits. – No coordination with the upstream grid
Ismail and Ahmed (2024), IEEE Access: review of cloud Li-ion BMS	Survey of cloud BMS frameworks	Limits of on-board BMS	+ Broad comparison. – Review only
Key technologies for cloud BMS, J. Electronic Materials (2024)	Survey: data, protocols, AI, digital twins, edge	Layer-by-layer comparison	+ Useful technology map. – Layers treated separately
AI + blockchain demand response, Scientific Reports (2024)	AI forecasting with blockchain trust	Scalability, privacy, real-time DR	+ Load balancing and trust together. – Consensus overhead hurts sub-second targets
Cloud AI framework for EV charging forecasting, Electronics (2026)	ARIMA + LightGBM microservices	Demand forecasting, station placement	+ Strong accuracy on real data. – Planning tool, not live control
Edge-intelligent EV charging coordination, Cluster Computing (2026)	CNN+LSTM, XGBoost on edge hardware	Cutting cloud latency	+ 27.6% better utilisation, 24.5% lower peak. – Edge hardware cost and complexity
Autonomous edge AI agents for EV charging (2026)	Domain AI agents at the edge, < 100 ms	Reliability of DC fast charging	+ Very fast response. – Hard to audit and coordinate grid-wide

Table 3: Literature survey summary

5. Analysis of existing research and research gap

Read in order, the works show a shift from the cloud as an **observer** (2020 to 2022: collect data, flag faults) to the cloud as a **decision-maker** (2023 to 2024: predictive analytics, condition-aware scheduling, trust). By 2026 the field splits: cloud-hosted ML for forecasting and planning on one side, and edge intelligence for split-second decisions on the other.

Three consistent findings stand out. Systems that rely on polling or batch processing struggle to react within seconds. Almost no work treats the utility side (OpenADR) and the charger side (OCPP) as one connected system. And security is usually added as a separate topic rather than designed in.

Research gap: none of the ten works combines (1) a real-time OpenADR to OCPP bridge, (2) a cloud-native pipeline fast enough to push new limits in under two seconds, (3) power allocation that adapts per vehicle and per moment, and (4) zero-trust security that can act on the grid itself. GridSync is designed to close this gap, and adds explainable AI decisions.

6. Challenges

- **Live connections for thousands of chargers:** devices on unreliable cellular links must stay reachable and reconnect cleanly.
- **Translating grid signals instantly:** an OpenADR event must be mapped to the right site and chargers and turned into OCPP limits within seconds.
- **Decisions in under 2 s:** the system must read live transformer load, active sessions and states of charge, then compute a fair and safe split.
- **Securing remote hardware:** every message must prove it comes from a genuine device, and a compromised device must be cut off without trusting it to cooperate.
- **Real-world constraints:** the AWS account's organisation policy limited the available regions and services, so the design had to adapt (Section 10.4).

7. AWS services used

Each service was chosen to solve a specific challenge. All are managed or serverless, so nothing runs idle on a provisioned server.

Service	Definition	Role in GridSync
AWS CloudFormation	Infrastructure as code: AWS resources defined in a template and deployed as a stack.	One template creates all 30 resources; one script deploys, one tears down.
AWS IoT Core	Managed service for securely connecting IoT devices over MQTT, with registry, X.509 authentication, policies and a rules engine.	9 things (8 chargers + transformer meter), mutual TLS, per-device policy, rule forwarding telemetry to SQS, OCPP commands to chargers.
Amazon SQS	Fully managed message queue that decouples producers and consumers and absorbs bursts.	Buffers telemetry between the IoT rule and the guard Lambda (batches of up to 100).
AWS Lambda	Serverless compute that runs code in response to events; pay per use.	4 functions: telemetry guard (AI), control steps (AI inference), OpenADR, dashboard API.
AWS Step Functions	Serverless workflow orchestration as a state machine with retries; Express type for high-volume, fast runs.	6-step decision workflow every ~2 s, synchronous for OpenADR events.
Amazon DynamoDB	Serverless NoSQL key-value database with millisecond latency.	Charger state table and a single-table site store (time series, hourly history, events) with TTL.
Amazon API Gateway	Managed service to create and secure HTTP/REST APIs in front of back-ends.	HTTP API: POST /openadr, GET /state, POST /mode, POST /demo.
Amazon S3	Durable object storage.	Website files, Lambda package and the trained model artifact.
Amazon CloudFront	Global content delivery network with HTTPS.	Serves the dashboard from a private S3 bucket via Origin Access Control.
AWS IAM	Identity and access management: users, roles and policies.	Least-privilege roles for Lambda, IoT rule and Step Functions.
Amazon CloudWatch	Monitoring: metrics, logs, dashboards.	GridSync dashboard (messages, executions, latency, errors) and Lambda logs.
AWS Budgets	Cost tracking with alerts.	$10/month budget with email alerts.

Table 4: AWS services: definition and role

8. Proposed and deployed architecture

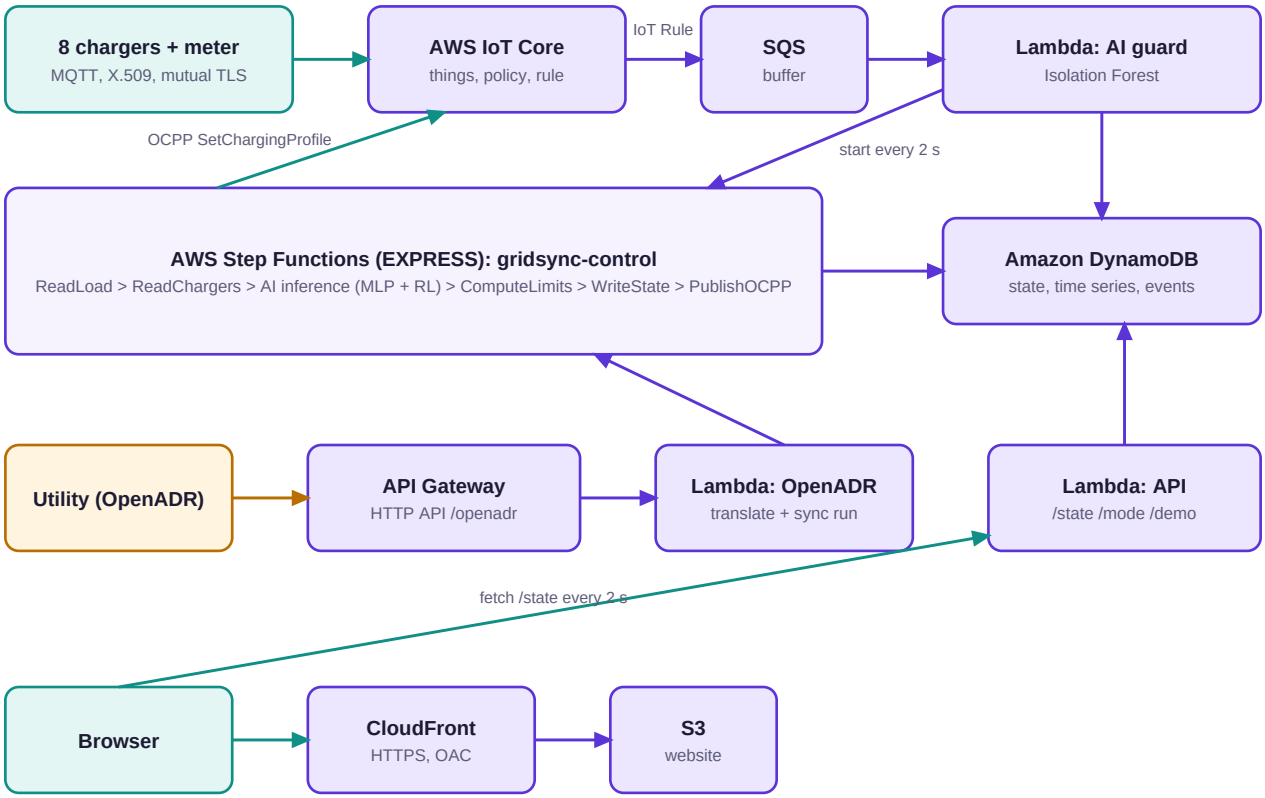


Figure 1: GridSync architecture as deployed in ap-southeast-2

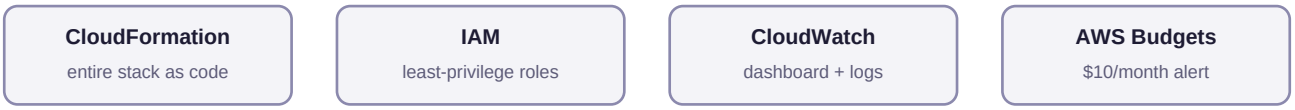


Figure 2: Cross-cutting services

8.1 Data flows

- **Telemetry:** every second each charger publishes voltage, current, power, limit and state of charge to `gridsync/<thing>/telemetry`. The IoT rule `SELECT *, topic(2) AS deviceId FROM 'gridsync/+/telemetry'` sends it to SQS, and the guard Lambda scores and stores it.
- **Control loop:** at most every 2 s the guard starts the Express state machine: ReadTransformerLoad, ReadChargerStatus, AIInference, ComputeLimits, WriteChargerState, PublishOcppCommands (OCPP SetChargingProfile on `gridsync/<thing>/cmd`).
- **Utility event:** POST /openadr on API Gateway invokes the OpenADR Lambda, which converts the requested reduction into an EV power cap and runs the workflow synchronously, so the HTTP reply reports the real end-to-end reaction time.
- **Security event:** when two of a charger's last three messages are anomalous, the guard sets its certificate INACTIVE, detaches its IoT policy and orders the trusted site controller to open that bay's contactor; the next control cycle reallocates power.
- **Dashboard:** the browser loads the site from CloudFront and polls GET /state every 2 s.

9. AI and machine-learning models

All three models are implemented from scratch in JavaScript (no ML libraries), trained on synthetic data generated from the same grid model, and evaluated on held-out data. The trained parameters are exported by a training job (`aws/scripts/export-models.js`, the role a SageMaker training job would play), stored in S3 and served inside Lambda.

9.1 Demand forecaster: neural network (supervised)

A multilayer perceptron (7 inputs, hidden layers of 16 and 12 tanh units, 1 output) trained with backpropagation and the Adam optimiser predicts next-hour site demand. Inputs: hour of day (sine and cosine), weekend flag, temperature, current load, load at the same hour yesterday and current EV demand. It was trained on 100 days and tested on 20 unseen days.

Metric (20 unseen days)	Value
MLP mean absolute error	148 kW
Seasonal-naive baseline MAE	224 kW
Error reduction	34%
R^2	0.81

Table 5: Forecaster accuracy

9.2 Charging agent: Q-learning (reinforcement learning)

A tabular Q-learning agent chooses what share of the transformer's headroom (55%, 70%, 80%, 90% or 100%) to give EV charging. Its state combines time of day, neighbourhood load, requested EV demand, the forecaster's predicted trend and today's peak (1,152 states). The reward is energy delivered, minus penalties for overloads, new demand peaks and waiting customers. It was trained on 6,000 simulated days and compared on 30 unseen days with the same random seed for every policy.

Policy (30 unseen days)	Overloads	EV peak (kW)	Energy (MWh)
Static per-charger limits	398	2000	165.8
Cloud rule (950 kW – base load)	85	731	165.4
GridSync RL agent	1	591	151.1

Table 6: Reinforcement-learning agent versus baselines

Against the fixed cloud rule, the agent cut overload events by 99% and the EV peak by 19%, while delivering 91% of the energy: it learns to trade a little charging speed for a safety margin against neighbourhood load spikes. When a live situation was never seen in training (all Q-values zero), the controller falls back to the nearest learned state and says so in its explanation.

9.3 Telemetry guard: Isolation Forest (unsupervised)

An Isolation Forest of 200 random trees scores every charger message. It is fitted on an unlabelled slice of the message stream (with about 2% unknown attacks), so no labelled attack data is needed. Its features are physics-informed: voltage deviation, the residual between reported power and voltage times current, compliance with the commanded limit and message cadence. A charger is quarantined when two of its last three messages exceed the threshold.

Metric (held-out messages)	Value
False-positive rate	0.0%

Metric (held-out messages)	Value
Overall attack recall (per message)	80%
False data injection	90%
Replay / message flooding	100%
Limit bypass + meter spoofing	51%

Table 7: Intrusion detector accuracy

Lesson from live testing. The first version, trained only on chargers drawing 40 to 100% of a limit of at least 15% of their rating, raised false alarms on the live system: cars waiting for their first limit (about 0 kW), heavily throttled sessions and cars drawing far less than allowed are all normal. The training data was widened to include these states, and a power-level feature that made high-power charging look like an outlier was removed. The false-positive rate on live-like states dropped to 0%, and a 60-second evening-rush soak test on AWS produced no false quarantines while a real attack was still caught. Limit-bypass detection is lowest because when a charger's limit is already near its rating, cheating changes almost nothing physically.

9.4 Explainable decisions

Every AI decision is stored with its explanation: the forecast and its main drivers (feature attributions by perturbation), the Q-value for every action, and a plain-language sentence. The dashboard shows this for every Step Functions run, which supports operator trust and auditability, a gap noted in the 2026 edge-agent literature.

10. Implementation on AWS

10.1 Infrastructure as code

The whole backend is `aws/template.yaml`. The deployed stack **gridsync** contains 30 resources:

Resource type	Count
ApiGatewayV2::Route	4
Lambda::Function	4
IAM::Role	3
ApiGatewayV2::Integration	2
DynamoDB::Table	2
Lambda::Permission	2
ApiGatewayV2::Api	1
ApiGatewayV2::Stage	1
Budgets::Budget	1
CloudFront::Distribution	1
CloudFront::OriginAccessControl	1
CloudWatch::Dashboard	1
IoT::Policy	1
IoT::TopicRule	1
Lambda::EventSourceMapping	1
S3::Bucket	1
S3::BucketPolicy	1
SQS::Queue	1
StepFunctions::StateMachine	1

Table 8: Resources in the deployed CloudFormation stack (exported from the live account)

Lambda function	Runtime	Memory	Trigger
gridsync-control-steps	nodejs22.x	1024 MB	Step Functions
gridsync-dashboard-api	nodejs22.x	512 MB	API Gateway
gridsync-openadr	nodejs22.x	512 MB	API Gateway
gridsync-telemetry-guard	nodejs22.x	512 MB	SQS

Table 9: Lambda functions

10.2 Data model (DynamoDB)

- **gridsync-charger-state** (key: deviceId): live status of each charger: reported power, state of charge, AI limit, anomaly score, recent flags, ACTIVE or QUARANTINED.
- **gridsync-site** (keys: pk, sk), single-table design: site state (mode, last decision and its timeline), the load time series, hourly aggregates used as forecaster inputs, and the event log. Time-series and event rows expire automatically after one day (TTL).

10.3 Deployment and devices

- `aws/scripts/deploy.ps1`: trains and exports the models, packages and uploads the Lambda code to S3, deploys the stack, creates the 9 IoT things with their own certificates, and publishes the website.
- `sim/charger-sim.js`: the edge simulator. Each of the 9 devices opens its own MQTT connection with its certificate, publishes telemetry every second and obeys SetChargingProfile commands. The transformer device also acts as the trusted site controller that opens or closes bay contactors.
- `aws/scripts/teardown.ps1` removes everything; `reset-demo.sh` restores a clean demo state.

10.4 Adapting to account constraints

The AWS account belongs to an AWS Organization whose service control policy (SCP) denies most services outside one region and denies Kinesis and Timestream entirely. The design was adapted rather than abandoned:

Original design	Deployed	Reason
Region ap-south-1 (Mumbai)	ap-southeast-2 (Sydney)	The SCP only allows this region
Kinesis Data Streams	IoT Rule → SQS → Lambda	Kinesis denied; SQS gives the same buffered, batched stream
Amazon Timestream	DynamoDB time series with TTL	Timestream denied, and closed to new customers
SageMaker real-time endpoint	Models trained offline, served in Lambda	No always-on cost; inference inside the 2 s path

Table 10: Design adaptations

11. Security design

- **Device identity:** every charger has its own X.509 certificate and private key and connects with mutual TLS; there are no shared passwords.
- **Least-privilege IoT policy:** policy variables restrict each device to connecting as itself, publishing only to its own telemetry topic and receiving only its own commands.
- **AI enforcement:** the Isolation Forest guard revokes a compromised device's certificate, detaches its policy and has the site controller cut its power within the same control cycle.
- **Least-privilege IAM:** Lambda can only reach the two project tables, the gridsync topics and the project state machine; the IoT rule can only send to one queue.
- **Private storage:** the website bucket blocks all public access and is readable only by CloudFront through Origin Access Control; all traffic is HTTPS.
- **Encryption:** DynamoDB, S3 and SQS encrypt data at rest by default; MQTT and HTTPS encrypt data in transit.

12. Cost analysis

Every component is pay-per-use. With the simulator running (9 devices, one message per second each, a control run every 2 s) the deployment costs roughly US$2 to $3 per day, dominated by IoT messages, DynamoDB writes and Lambda invocations. With the devices stopped, cost falls to almost zero (storage only). An AWS Budget emails the team at 80% of a $10 monthly limit. Old telemetry expires automatically through DynamoDB TTL, so storage does not grow. At larger scale the same design scales horizontally with no servers to provision, unlike an always-on fleet sized for the evening

peak.

13. Results and evaluation

All live figures below were measured on the deployed AWS system while recording the demo video.

Objective	Result	Status
O1: reaction under 2 s	Control decision 0.16 s; OpenADR –30% reached all chargers in 0.55 s	Met
O2: lower peak demand	RL agent: EV peak –19% vs fixed rule; live EV load held to the AI budget during the rush	Met (simulation)
O3: transformer < 100%, zero trips	Load stayed below 100% in AI mode; no trips during rush, OpenADR and attack demos	Met
O4: 99.99% uptime	Managed serverless services; devices reconnect automatically through IoT Core	By design (not measured long-term)
O5: zero-trust security	Attack detected in about 1 s, certificate revoked, bay isolated; 0% false alarms in soak test	Met

Table 11: Results against objectives

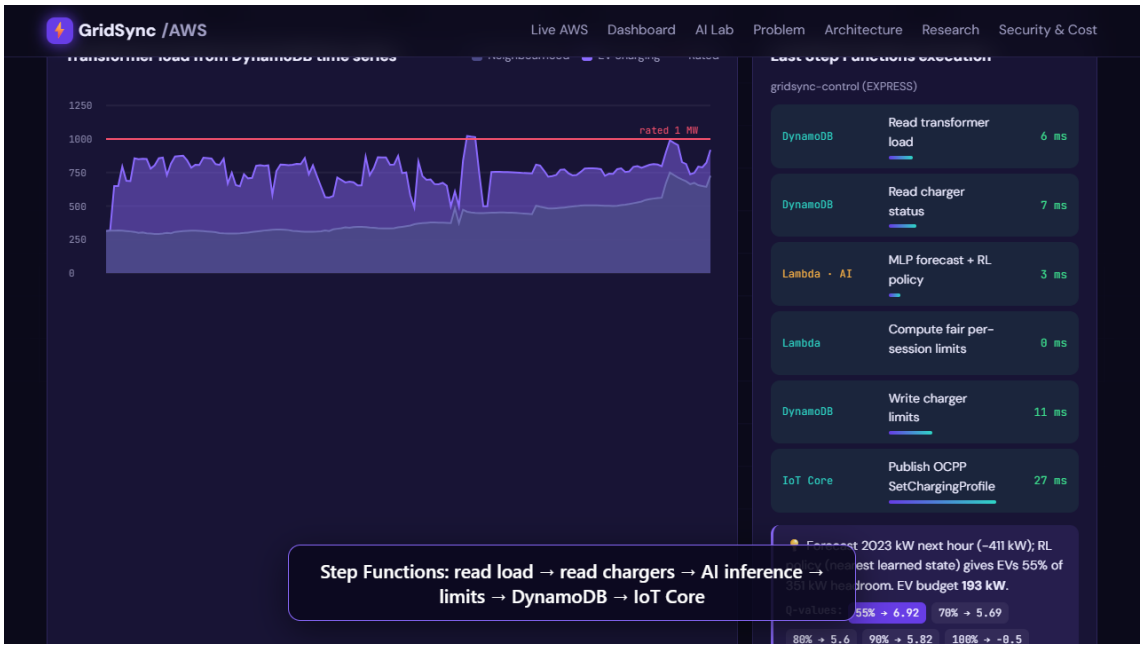


Figure 3: Live control room: transformer load from the IoT meter, Step Functions timeline and the AI's explanation

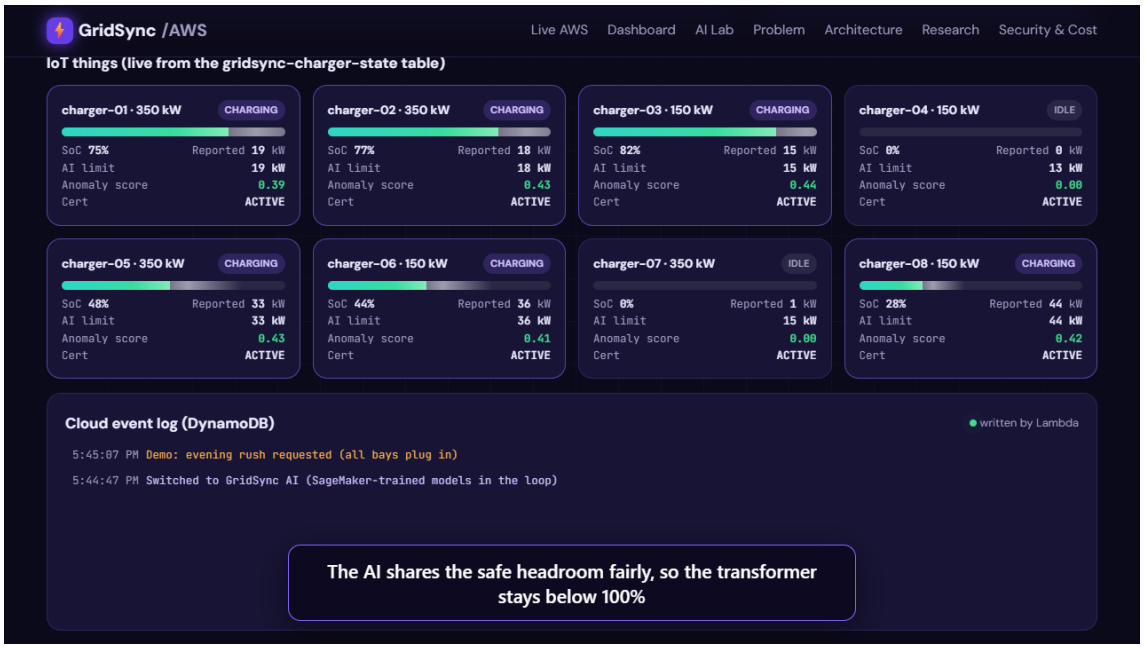


Figure 4: Evening rush: all eight chargers active and sharing the AI-computed budget

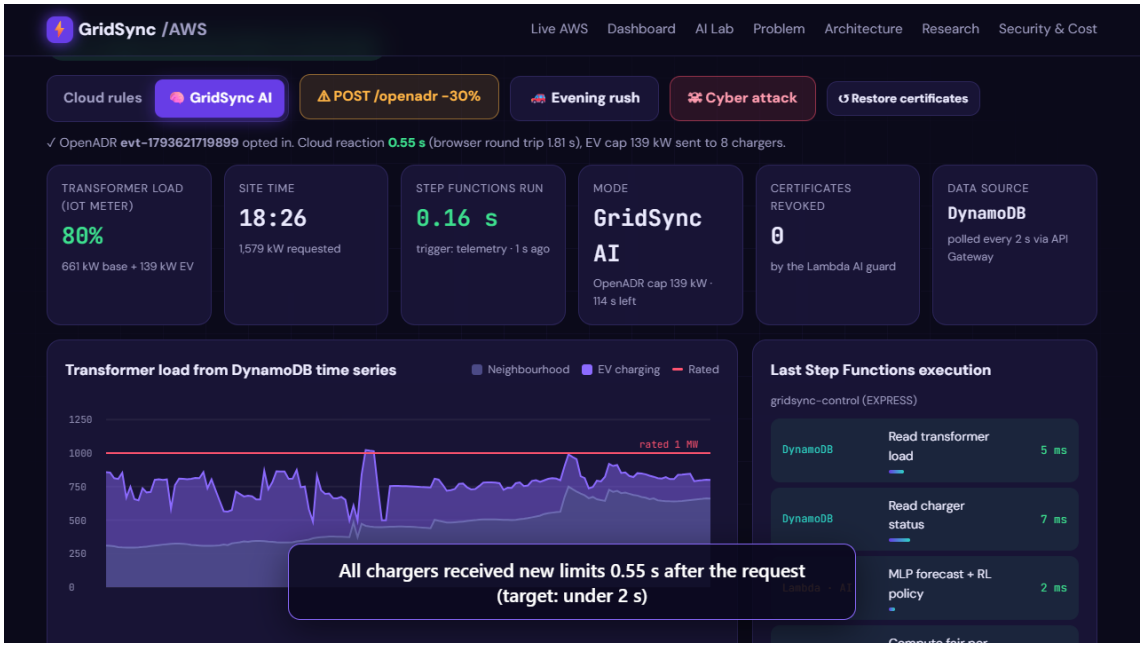


Figure 5: OpenADR –30% request answered by the cloud in 0.55 s

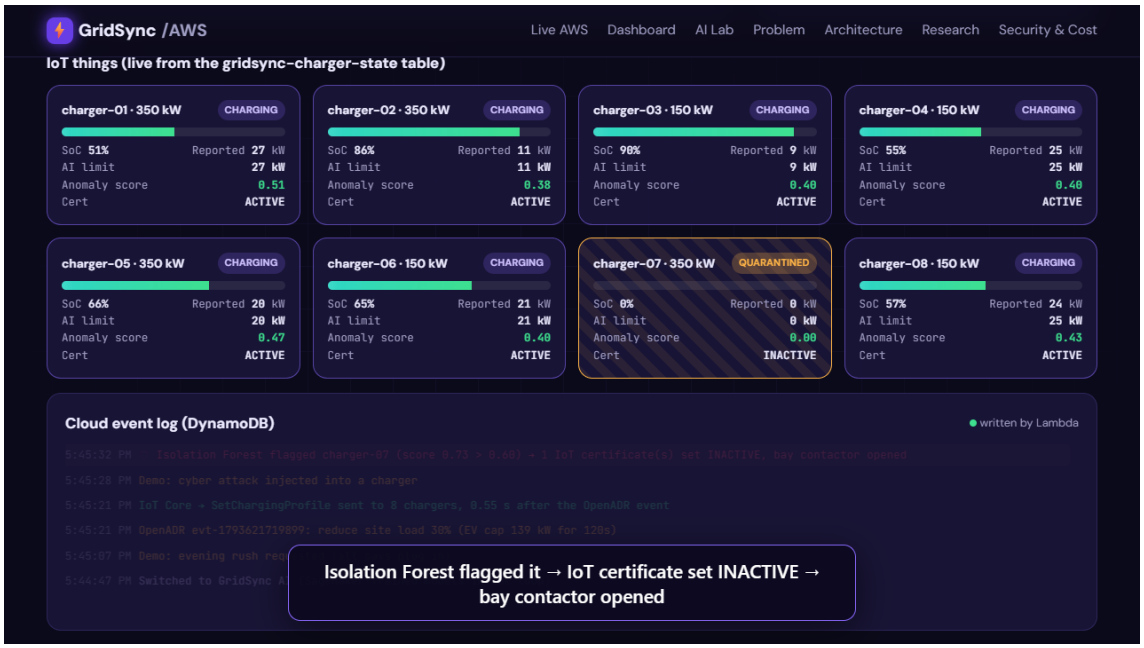


Figure 6: Cyber attack: the compromised charger is quarantined and its certificate set INACTIVE

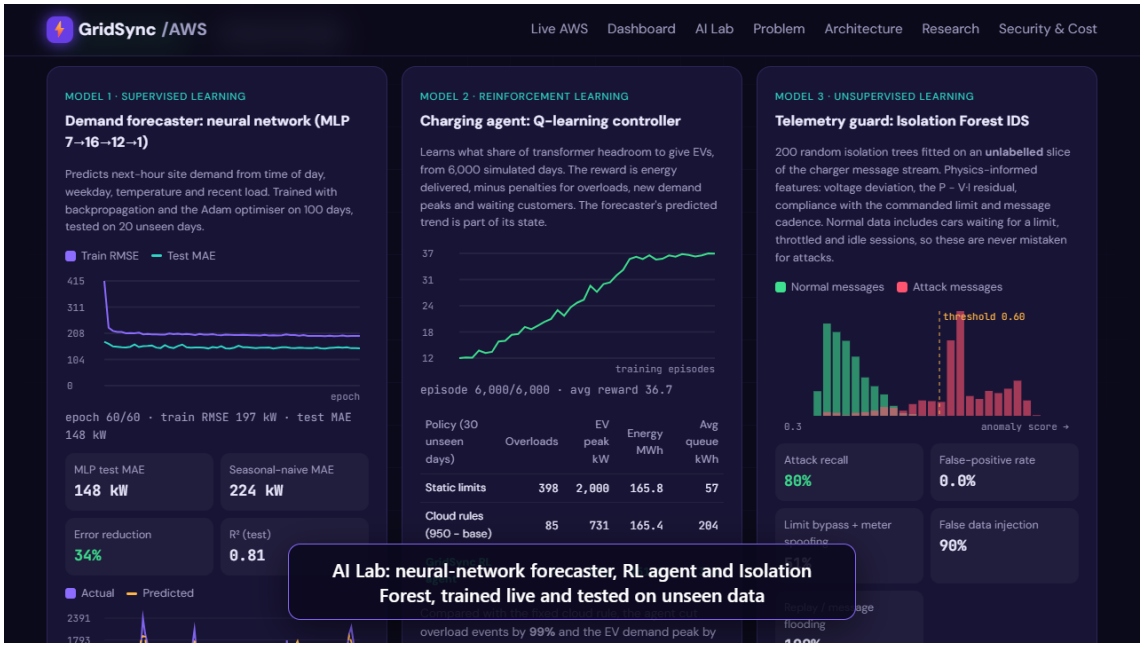


Figure 7: AI Lab: the three models trained and evaluated on held-out data

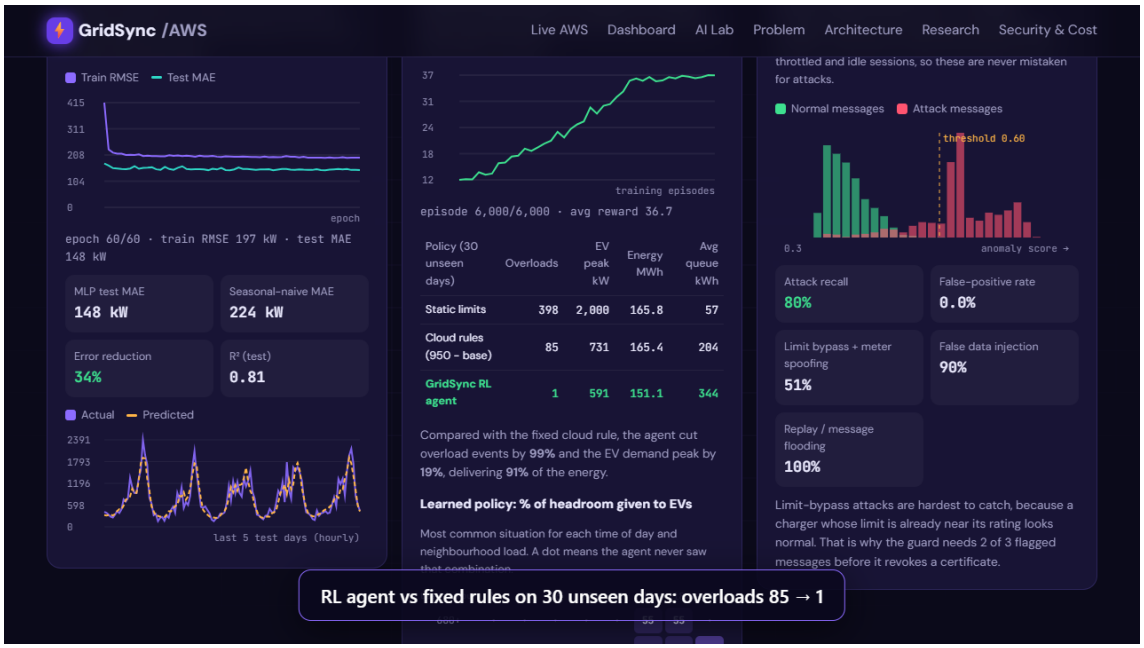


Figure 8: Reinforcement-learning agent compared with static limits and fixed cloud rules

14. Novelty

- **Forecast-informed reinforcement-learning control:** a neural forecast feeds an RL agent that learns its own safety margin, instead of a hand-tuned limit.
- **Security that acts on the grid:** an unsupervised, physics-informed detector revokes the device's identity in IoT Core and reallocates its power in the same cycle, joining the security and grid-control loops.
- **Explainable decisions:** every action is logged with Q-values, forecast drivers and a plain-language reason.
- **OpenADR to OCPP in under 2 s, fully serverless:** deployed as code, scaling from one site to thousands of chargers with no servers to manage.

15. Limitations and future work

- Grid, vehicles and attacks are simulated; a pilot with real OCPP 2.0.1 chargers and real utility OpenADR 3.0 events is the next step.
- Telemetry uses SQS because of account policy; with Kinesis available, ordered per-device streams would allow longer real-time analytics windows.
- The RL agent is tabular; a deep RL policy (trained on SageMaker) could handle many sites and finer actions.
- Limit-bypass attacks near a charger's full rating are hard to detect from telemetry alone; adding the transformer's measured current per feeder would close this gap.
- Long-term availability (99.99%) was designed for but not measured over weeks.

16. Conclusion

GridSync shows that a fully serverless AWS architecture can coordinate EV fast chargers with the grid fast enough to prevent overloads. Deployed as a single CloudFormation stack on a real account, it bridges OpenADR and OCPP in about half a second, recomputes every charger's limit every two seconds with a neural-network forecast and a reinforcement-learning policy, and isolates a compromised charger within about a second. Testing on the live system also exposed and fixed a real false-alarm problem in the intrusion detector, which strengthened the final design. The result is a system that is fast enough to protect the grid, flexible enough to use spare capacity, and secure by design.

References

[1] Gao et al., remote monitoring and fault detection for EV charging devices, 2020.

[2] Concept review of a cloud-based smart battery management system for lithium-ion batteries, MDPI Batteries, 2022.

[3] Karnehm et al., cloud-based framework for scalable, centralised battery analytics, 2023.

[4] Saner, Saha and Srinivasan, charge-curve and BMS-aware optimal charging scheduling framework, IEEE Transactions on Intelligent Transportation Systems, 2023.

[5] Ismail and Ahmed, a comprehensive review of cloud-based lithium-ion battery management systems for EV applications, IEEE Access, 2024.

[6] A brief review of key technologies for cloud-based battery management systems, Journal of Electronic Materials, 2024.

[7] Optimizing demand response and load balancing in smart EV charging networks using an AI-integrated blockchain framework, Scientific Reports, 2024.

[8] Cloud-based AI framework for EV charging forecasting and infrastructure optimization, Electronics (MDPI), 2026.

[9] Edge-intelligent EV charging coordination for grid load balancing and renewable integration, Cluster Computing, 2026.

[10] Autonomous edge-deployed AI agents for electric vehicle charging infrastructure management, 2026.

[11] F. T. Liu, K. M. Ting and Z.-H. Zhou, Isolation Forest, IEEE International Conference on Data Mining, 2008.

[12] Amazon Web Services documentation: AWS IoT Core, Amazon SQS, AWS Lambda, AWS Step Functions, Amazon DynamoDB, Amazon API Gateway, Amazon CloudFront, AWS CloudFormation.

Appendix A: repository layout and how to run

Path	Contents
aws/template.yaml	CloudFormation template for the whole backend
aws/lambda/	guard.js (AI guard), control.js (workflow steps), openadr.js, api.js, lib.js
aws/scripts/	export-models.js (training job), deploy.ps1, teardown.ps1, reset-demo.sh
sim/charger-sim.js	Edge simulator: 8 chargers + transformer over MQTT and mutual TLS
ml.js, ai.js	From-scratch MLP, Q-learning and Isolation Forest; grid model and training
index.html, app.js, aws-live.js	Dashboard: live AWS control room, simulation and AI Lab
docs/	This report, pitch, presentation script and demo video

Table 12: Repository layout

Run the demo

```
cd sim
node charger-sim.js
# then open https://dye305b97k5t7.cloudfront.net and choose Live on AWS

# redeploy from scratch
.\aws\scripts\deploy.ps1 -BudgetEmail <email>
# remove everything
.\aws\scripts\teardown.ps1
```